SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – TECHNICAL PRESENTATION

Course Code:	CSA0620	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Technical Presentation
Weightage:	10% of CIA	**Total Marks:**	100 (1 Question × 100 Marks)
CO1 Statement:	*Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)*		
Student Name:	Shree Thilak Muthukrishna	**Reg. No.:**	192572046
Section / Batch:	B.E. CSE AI	**Date:**	23-07-2026

Instructions to Students

- Answer the given question through a technical presentation. Total Marks: 100.
- Clearly define the problem and explain the concept of algorithm growth functions.
- Present comparative analysis using appropriate representations such as graphs, tables, or charts.
- Use suitable tools (PowerPoint, visualization tools, or software) to support your explanation.
- Ensure technical accuracy, logical flow, and proper interpretation of results.
- Maintain clarity in communication, proper slide organization, and professional delivery.
- Time management, confidence, and audience engagement will be considered during evaluation.

Question Type	Focus Area	Questions	Marks
Technical Presentation	Analyze and compare growth functions using mathematical techniques and asymptotic concepts and with graphical and visual interpretation	Q1	100
	GRAND TOTAL:	100	

SECTION A – Technical Presentation

Q1.Algorithm Comparison Using Visualization

Compare two algorithms of different complexities (e.g., $O(n^2)$ vs $O(n \log n)$) using graphical visualization. Clearly define the problem and dataset assumptions. Use tools like plotting graphs or tables to show differences. Analyze the results and justify why one algorithm outperforms the other. Present conclusions in a clear and professional manner.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Marks Evaluation:

Question No.	Technical Content Accuracy (30%)	Problem Identification (25%)	Use of Tools (20%)	Communication (15%)	Professionalism (10%)	Total Marks (100)
Q1						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SAVEETHA SCHOOL OF ENGINEERING
Affliated to AICTE | IET-UK Accreditation

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Engineer to Excel

ALGORITHM PERFORMANCE ANALYSIS

A Comparative Evaluation of Quadratic $O(n^2)$vs. Linear $O(n \log n)$Computational Scalability Across Expanding Datasets

Presenter: Shree Thilak Muthukrishna
Register No: 192572046
Course: CSA0620 – Design and Analysis of Algorithms

Problem Statement

Core Challenge

- In modern computational engineering, data growth rapidly outpaces memory bandwidth and processor cycles. Selecting an inefficient algorithm leads to exponential delays, server timeouts, and infrastructure bottlenecks.

Core Objectives

- Analyse execution time scaling as input array size (n) expands from small arrays to enterprise-scale workloads.
- Quantify exact operational overhead differences between iterative brute-force swapping and divide-and-conquer paradigms.
- Determine the crossover point where logarithmic algorithmic efficiency overcomes recursive setup overhead.

Dataset & Benchmark Assumptions

- **Input Distribution:** Uniformly distributed, randomly shuffled numeric datasets processed in RAM.
- **Evaluation Metrics:** Total comparison count, memory swaps/move operations, and growth trajectory.

Algorithm Overview

Bubble Sort (Quadratic Paradigm)

- An iterative sorting algorithm that repeatedly steps through an array, compares adjacent elements, and swaps them if they are out of order.
- **Time Complexity:** Best case $O(n)$(with optimized flag), Average/Worst case $O(n^2)$.
- **Space Overhead:** $O(1)$strictly in-place auxiliary space.

Merge Sort (Linear Paradigm)

- An efficient divide-and-conquer algorithm that recursively breaks an array into single sub-elements and merges them in sorted order.
- **Time Complexity:** Guaranteed $O(n \log n)$across best, average, and worst cases.
- **Space Overhead:** $O(n)$auxiliary buffer memory allocation required.

Algorithm / Pseudocode

Bubble Sort Procedure

```
procedure BubbleSort(A: List)
    n = length(A)
    for i = 0 to n - 1 do
        swapped = false
        for j = 0 to n - i - 2 do
            if A[j] > A[j+1] then
                swap(A[j], A[j+1])
                swapped = true
        if not swapped then break
```

Merge Sort Procedure

```
procedure MergeSort(A: List)
    if length(A) <= 1 return A
      mid = length(A) / 2
      left = MergeSort(A[0 .. mid-1])
      right = MergeSort(A[mid ..
     length(A)-1])
    return Merge(left, right)
```

Working Example (Dry Run)

Input Dataset: [38, 27, 43, 3]

1. Bubble Sort Step Trace

- **Pass 1:** Compare (38, 27) →Swap; Compare (38, 43); Compare (43, 3) →Swap
  Array State: [27, 38, 3, 43]
- **Pass 2:** Compare (27, 38); Compare (38, 3) →Swap
  Array State: [27, 3, 38, 43]
- **Pass 3:** Compare (27, 3) →Swap
  Final State: [3, 27, 38, 43] *(Sorted in 6 total comparisons)*

Working Example Dry Run

2. Merge Sort Step Trace

- **Divide Phase:** Subdivide into equal halves →[38, 27] and [43, 3]
- **Conquer Phase:** Base elements →[38], [27], [43], [3]
- **Merge Phase:** Two-pointer recombine:
    - Pairs merged →[27, 38] & [3, 43]
    - Final merge →[3, 27, 38, 43]

Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case	Recurrence Relation
Bubble Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$T(n) = T(n-1) + O(n)$
Merge Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$T(n) = 2T(n/2) + O(n)$

Complexity Analysis

Master Theorem Proof for Merge Sort

For recurrence $T(n) = 2T(n/2) + n$:

- **Parameters:** $a = 2, b = 2$
- **Critical Exponent:** $n^{\log_b a} = n^{\log_2 2} = n^1 = n$
- **Work Function:** $f(n) = \Theta(n)$
- **Conclusion:** Since $f(n) = \Theta\left(n^{\log_b a}\right)$, **Case 2** of the Master Theorem applies: $T(n) = \Theta(n \log n)$.

Comparative Analysis

Metric / Feature	Bubble Sort $O(n^2)$	Merge Sort $O(n \log n)$	Performance Impact & Scale Factor
Data Comparisons	High: $\sim n^2/2$	Low: $\sim n \log_2 n$	Merge Sort performs $>$ 99% fewer comparisons at $n = 10{,}000$
Data Swaps / Moves	High: $\sim n^2/2$	Moderate: $\sim n \log_2 n$	Bubble Sort generates unsustainable disk / RAM write overhead
Cache Locality	Sequential access	Non-contiguous recursion	Bubble Sort exhibits higher cache hits, offset by huge operations
Auxiliary Space	$O(1)$ Zero allocation	$O(1)$ Buffer allocation	Merge Sort requires supplementary memory proportional to dataset size.
Data Stability	Stable	Stable	Both algorithms preserve relative positioning of equal key items

Applications

Optimal Scenarios for $O(n^2)$Bubble Sort

- **Embedded Systems & Microcontrollers:** Ultra-low memory devices (e.g., 2KB SRAM microcontrollers) where extra buffer space cannot be allocated.
- **Small / Near-Sorted Arrays:** Educational modules or minimal lists ($n < 15$) where function call overhead exceeds sorting work.

Optimal Scenarios for $O(n \log n)$Merge Sort

- **Production Database Indexing:** Relational engines (PostgreSQL, MySQL) utilize Merge/Timsort variations for disk/memory sorting.
- **Distributed Computing Pipelines:** Large-scale Big Data engines (Apache Spark, Hadoop) during sort-merge join processing.
- **Genomic Sequence Alignment:** Processing millions of biological DNA reads rapidly.

Results & Discussion

Empirical Execution Findings

- **Small Scale ($n < 50$):** Difference in execution time is practically negligible (< 0.1 ms).
- **Large Scale Breakpoint ($n = 100{,}000$):**
    - Bubble Sort requires $\sim 10^{10}$ operations (~ 100 sec CPU runtime).
    - Merge Sort executes in $\sim 1.7 \times 10^{6}$ operations (~ 0.02 sec CPU runtime).
- **Speedup Multiplier:** Merge Sort yields a $> 5{,}000\times$ execution advantage on large datasets.

Conclusion & References

Core Conclusions

- Algorithmic growth complexity ($O(n \log n)$) dominates hardware speedups when handling expanding real-world datasets.
- Merge Sort offers predictable execution consistency across worst, average, and best-case conditions.
- Modern hybrid algorithms (e.g., Timsort) combine Insertion Sort efficiency on small blocks with Merge Sort scalability.

Academic References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.


Problem Statement
Core Challenge
In modern computational engineering, data growth rapidly outpaces memory bandwidth and processor cycles. Selecting an inefficient algorithm leads to exponential delays, server timeouts, and infrastructure bottlenecks.
Core Objectives
Analyse execution time scaling as input array size (n) expands from small arrays to enterprise-scale workloads.
Quantify exact operational overhead differences between iterative brute-force swapping and divide-and-conquer paradigms.
Determine the crossover point where logarithmic algorithmic efficiency overcomes recursive setup overhead.
Dataset & Benchmark Assumptions
Input Distribution: Uniformly distributed, randomly shuffled numeric datasets processed in RAM.
Evaluation Metrics: Total comparison count, memory swaps/move operations, and growth trajectory.
GPS Map Camera
Google
Mevalurkuppam, Tamil Nadu, India 🇮🇳
Eee Department, Mevalurkuppam, Tamil Nadu 602105, India
Lat 13.026556° Long 80.016224°
Thursday, 23/07/2026 11:20 AM GMT +05:30